Biblioteca com self-checkout

Arquitetura implementada, regras e guia de validação do primeiro backend.

UMA API. UM ACERVO. DOIS CANAIS.

Empréstimos no balcão e autoempréstimo pelo leitor usam o mesmo banco, as mesmas regras e a mesma proteção transacional.

01 O que esta entrega contém

API em **C# / ASP.NET Core 10**, persistência real em **SQL Server**, catálogo, usuários e perfis, empréstimos, autoempréstimo, devoluções parciais, configurações e auditoria. O modelo acompanha os conceitos das fotos da aula: PK/FK, relação N:N, transações, procedures e log com NEWID().

02 O significado de self-checkout

O leitor entra com a própria conta e confirma a retirada dos exemplares por seus códigos de tombo. A operação devolve um comprovante com itens e prazo. Não é uma compra e não envolve pagamento. A devolução permanece com a equipe.

03 O que ainda não foi desenvolvido

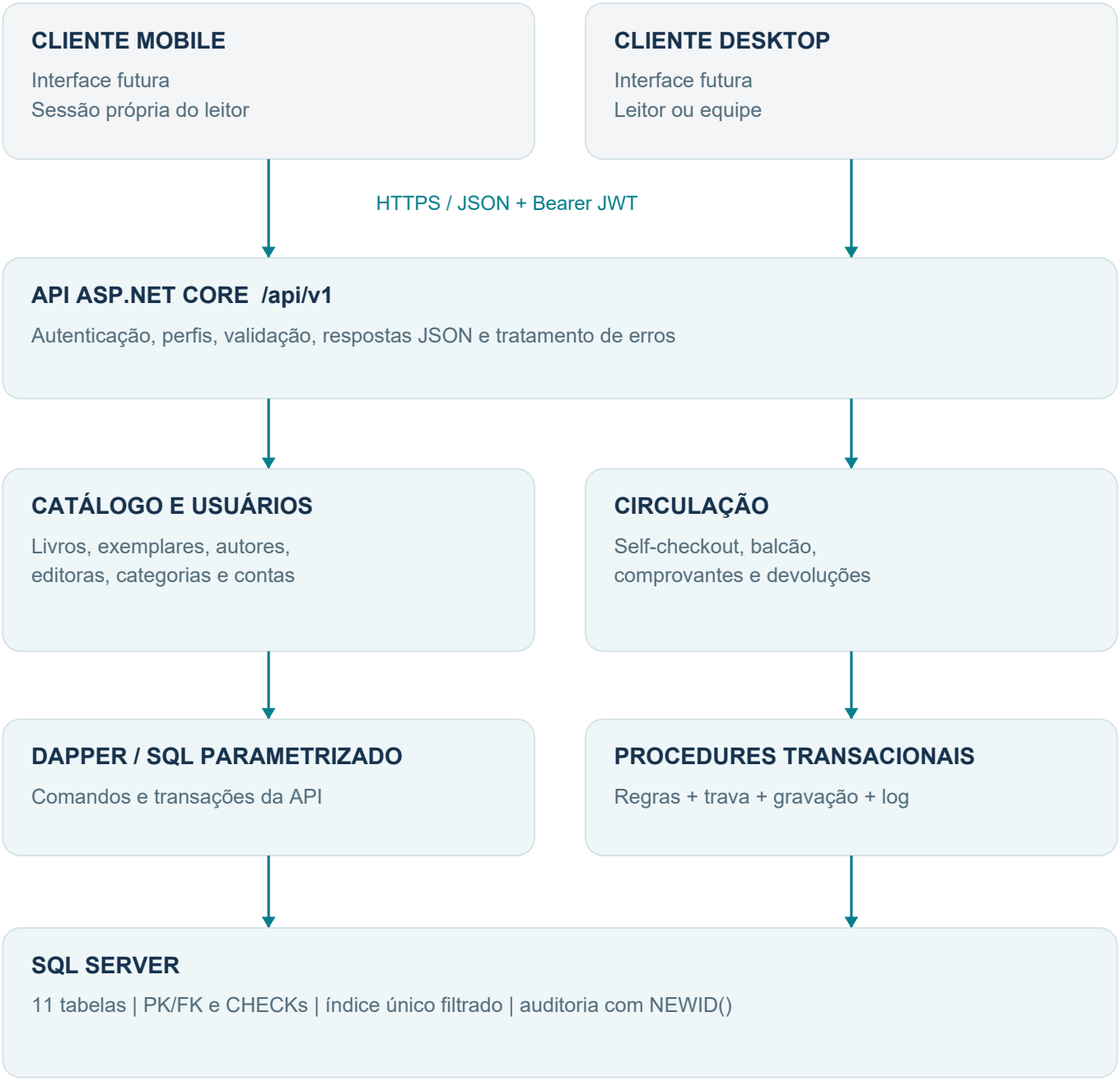
A interface para celular e desktop será decidida na próxima etapa. Não há telas, leitura por câmera, etiquetas impressas, RFID, aplicativo instalável ou publicação na internet nesta versão. Os dois tipos de cliente poderão consumir esta API.

VALIDADO EM SQL SERVER REAL

18 testes HTTP passaram e **5 invariantes de integridade** foram conferidas. A compilação Release terminou sem erros ou avisos.

Diagrama de arquitetura

Monólito modular: um processo de API, com responsabilidades separadas por módulo.

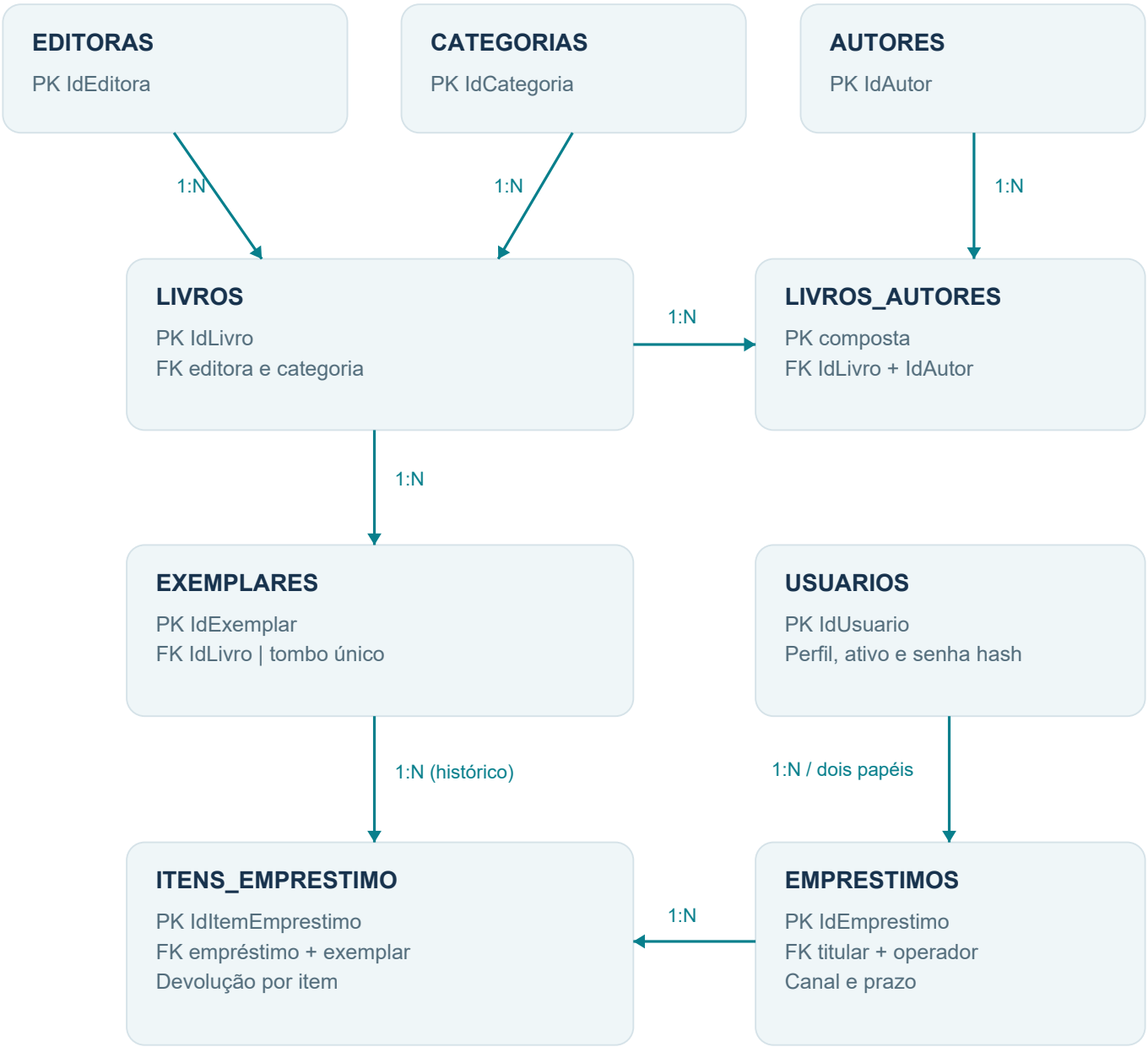


Fronteira de segurança: somente a API acessa o banco. As telas não escolhem a identidade do autoempréstimo nem alteram diretamente a disponibilidade.

Estado atual: execução local em HTTP no endereço de loopback. HTTPS, domínio, CORS e credenciais restritas precisam ser configurados antes de publicar.

Modelo relacional

Visão das 11 tabelas. Os arquivos SQL são a definição completa de campos, chaves e restrições.



TBL_LOG_SISTEMA

PK GUID (NEWID)
FK usuário opcional + evento e instante

CONFIGURACOES

Uma linha: limite, prazo, atrasos e self-checkout

Log: usuário 1:N eventos; evento pode não ter usuário. Configuração não possui FK.

Fluxo do self-checkout

O código de tombo identifica o exemplar físico. ISBN identifica a edição do livro, não sua cópia.

1 **Autenticar**

POST /api/v1/auth/login. O token identifica o leitor ativo.

↓

2 **Conferir os códigos**

Consultar o exemplar por código e montar a lista para confirmação.

↓

3 **Confirmar a retirada**

POST /api/v1/self-checkout/emprestimos recebe apenas codigosExemplares.

↓

4 **Revalidar no banco**

Verificar identidade, política, atrasos, limite e disponibilidade atual.

↓

5 **Gravar e devolver o comprovante**

Empréstimo + itens + disponibilidade + log. Sucesso: HTTP 201.

Limite / prazo	3 exemplares pendentes e 14 dias; parâmetros provisórios, alteráveis pelo administrador.
Atomicidade	Todos os itens são confirmados ou nenhum. Qualquer falha desfaz a transação.
Concorrência	Trava transacional e índice único impedem duplicidade e protegem o limite total.
Devolução	Equipe confere o exemplar; pode devolver parte dos itens. Finaliza após o último.

Executar e integrar

O pacote contém código-fonte, solução .NET, scripts SQL, requisições e testes reproduzíveis.

Início local

Pré-requisitos: .NET SDK 10, SQL Server, sqlcmd e PowerShell. Abra um terminal na pasta do projeto e execute **./scripts/Start-Local.ps1**. O script prepara o banco e, na primeira execução, pede a senha do administrador. Não existe senha padrão. O servidor fica em **http://127.0.0.1:5180**.

Acervo e primeiro empréstimo

A demonstração inclui 3 títulos fictícios e 6 exemplares. Faça login como admin, cadastre uma conta LEITOR e autentique-se com ela. Envie **{"codigosExemplares":["DEMO-BD-001"]}** ao endpoint de self-checkout. Use o accessToken no cabeçalho Authorization: Bearer. Exemplos editáveis estão em **docs/requisicoes.http**.

Contrato da API	/openapi/v1.json, importável em ferramentas de teste. /health verifica o banco.
Leitor	Consulta catálogo e política, faz autoempréstimo e consulta o próprio histórico/comprovante.
Bibliotecário	Gerencia acervo, cadastra leitores, registra empréstimos no balcão e devoluções.
Administrador	Inclui as funções de equipe, gerencia perfis/ativação, política e consulta auditoria.
Erros HTTP	400 dados; 401 sessão; 403 permissão; 404 inexistente/inacessível; 409 conflito; 422 regra de negócio; 429 limite de login.

Cuidados de segurança e publicação

Senhas são hashes PBKDF2, e tokens duram 60 minutos. A conta ativa é revalidada em cada chamada protegida. Em Development, a chave JWT pode ser efêmera. Antes de publicar: usar HTTPS, chave persistente forte, credencial SQL restrita, certificado confiável, origens e host explícitos. Esta entrega não foi publicada.

Validação e próximos marcos

Resultados observados em 01/10/2026. Estes testes cobrem o backend, não a futura interface.

Compilação Release	Concluída: zero erros e zero avisos; restauração com dependências travadas.
Testes HTTP	18 de 18 aprovados: autenticação, autorização, titularidade, regras e contratos.
Concorrência	Duas pessoas / mesmo exemplar: só uma confirma. Mesmo leitor / dois pedidos: limite total preservado.
Rollback	Códigos inválidos, itens já emprestados e autor inexistente não deixam gravação parcial.
Circulação	Atrasos, manutenção, bloqueio do self-checkout, retorno parcial/final e novo empréstimo.
Integridade SQL	5 verificações: exclusividade, disponibilidade, cabeçalho/itens, titularidade do autoempréstimo e FKs.

Como reproduzir

Execute **dotnet build Biblioteca.slnx** e depois **./scripts/Test-Integration.ps1 -Python python**. O script cria banco exclusivo, usa credenciais efêmeras e encerra a API de testes. O banco permanece disponível para consulta. A execução registrada usou **BibliotecaTest_20261001091201**. O teste de atraso altera somente sua própria massa de teste.

Limitações conhecidas

A trava global de circulação é apropriada para o MVP acadêmico, mas deve ser reavaliada para alto volume. Ainda não há testes de carga, recuperação de senha, renovação, reserva, multas ou confirmação idempotente. Em falha de rede após confirmar, consulte o histórico antes de repetir a operação.

Referência e continuidade

O DDL completo está em **database/001-schema.sql**; as transações, em **database/002-procedures.sql**. O README explica instalação e configuração. Próximos marcos: decidir a interface responsiva, implementar login e autoatendimento e validar as regras com o professor. Este PDF atualiza o planejamento v0.1.